



1125

BHARATIYA ANTARIKSH HACKATHON 2026



Team Name : CanopyBreakers

Team Leader Name : SATHYAGITH V

Problem Statement : Route Resilience

Team Members

Team Leader:

Name:SATHYAGITH.V
College:IIT BHUBANESWAR

Team Member-1:

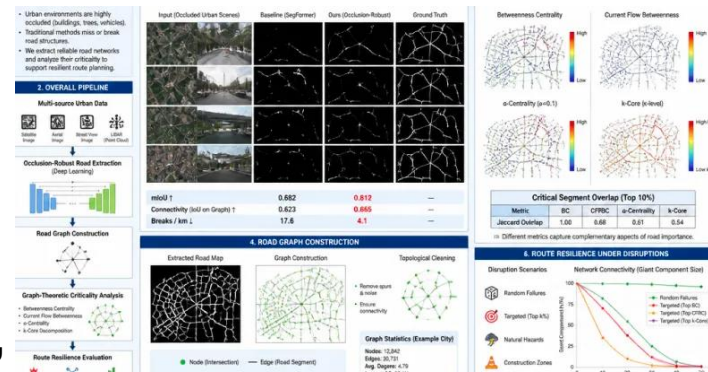
Name:NAMAN AGRAWAL
College:IIT BHUBANESWAR

Team Member-2:

Name:NIKHIL RAI
College:IIT BHUBANESWAR

Challenges

- Indian cities like Bengaluru face **fragmented and disconnected** road networks in spatial models.
- Satellite road extraction fails under **trees, shadows, and clouds** — roads disappear in the mask.
- Broken masks have **no usable topology**, so they cannot support disaster response, routing, or traffic planning.
- We need a pipeline that **extracts roads through occlusions** and turns them into a **connected graph** to find bottlenecks and simulate failures.



Approach

We separate the problem into two tasks: first, accurately detect roads from satellite images, and then build a connected road network for analysis.

*We trained both **UNet++ (ResNet34 + scSE attention)** and **DeepLabV3+** on the **DeepGlobe** dataset. UNet++ served as the benchmark model, while **DeepLabV3+** achieved better performance and was selected as the final model. Occlusion-aware augmentation was used to improve road detection under trees and shadows*

*The segmented road mask is converted into a road graph. Broken road segments are repaired using **Union-Find**, **Minimum Spanning Tree (MST)**, and geometric checks based on distance and direction to preserve realistic road connectivity.*

*We compute **betweenness centrality** on the largest connected road network to identify critical junctions. Road path lengths are used as edge weights to produce realistic importance scores.*

***Phase I** (model training and inference) was performed on an **NVIDIA T4 GPU**, while **Phases II, III, and IV** (graph construction, criticality analysis, and dashboard development) were executed on the **CPU**.*

Solution

Our solution has four main phases that work together to find critical road junctions and test road network resilience.

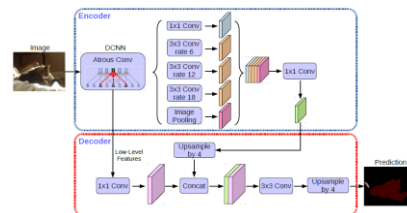
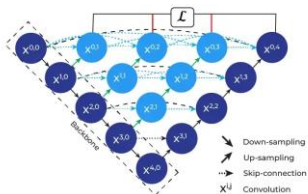
Phase I: We trained both **UNet++ with ResNet34 and scSE attention** and **DeepLabV3+** for road segmentation. UNet++ was used as a strong benchmark, while DeepLabV3+ gave better final results. We also used occlusion-aware augmentation to detect roads under trees, shadows, and gaps.

Phase II: The road mask is converted into a road graph. Broken road segments are repaired using **Union-Find** and **MST-based graph healing**.

Phase III: We use **betweenness centrality** to find important junctions. Then we simulate removing those junctions to see how much the road network is affected.

Phase IV: A **React dashboard** shows the heatmap and lets users test road failure scenarios on the map.

Tools and Technology Used



$$DSC(A,B) = \frac{2 \times \text{predicted} \cap \text{ground truth}}{\text{predicted} \cup \text{ground truth}}$$

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$



Tools and Technology Used

UNet++ — Uses nested dense skip connections to improve feature fusion and produce more accurate road segmentation with sharp boundaries.

scSE Attention — Enhances important road features by applying both channel-wise and spatial attention during decoding.

DeepLabV3+ with ASPP — Captures multi-scale contextual information using parallel dilated convolutions, improving segmentation in complex scenes.

ResNet34 Encoder — Extracts rich image features using an ImageNet-pretrained backbone, providing a strong foundation for segmentation.

Segmentation Models PyTorch (SMP) — Provides a unified framework for implementing and fairly comparing different segmentation architectures.

Albumentations — Applies realistic data augmentation techniques to improve the model's robustness to shadows, occlusions, and other real-world variations.

scikit-image — Converts segmented road masks into one-pixel-wide centerlines for accurate graph generation.

NetworkX — Builds the road network graph, repairs disconnected road segments, and analyzes network connectivity and critical junctions.

Tools and Technology Used

React + Vite — Builds a fast, interactive dashboard where planners explore the metrics, the gatekeeper list, and failure simulations.

Leaflet — Maps the road network, critical junctions, and the criticality heatmap onto a live satellite basemap.

JSON handoff — Keeps the machine-learning processing fully separate from the interface, so each part can run and scale on its own.

DeepGlobe Dataset — Trains on open global imagery and runs on ISRO Earth-observation tiles, so the same approach scales to Bengaluru.

Docker — Packages each processing stage in its own container, so batch jobs run the same way every time and on any machine.

Apache Airflow — Automates the full city refresh end to end, so the whole pipeline reruns on a set schedule without manual steps.

How different is it from any of the other existing ideas?

- Most road-extraction work stops at the pixel mask, treating visible-asphalt accuracy as the final result
- Broken segments under tree cover are accepted as a limitation of the imagery
- Route Resilience is a full mobility pipeline, not a segmentation demo
- The mask becomes a routable graph, analyzed for bottlenecks and shown to planners in an interactive dashboard
- We treat the mask as an intermediate product that exists to support network connectivity and downstream decision-making

How different is it from any of the other existing ideas?

Segmentation-only systems

They output white pixels on a map. There is no graph, no routing logic, and no way to ask what happens if a junction floods or closes.

Training without occlusion simulation

Most models use only standard flips and brightness changes. They are not trained to handle missing road patches under trees and shadows, so predictions break exactly where Indian urban imagery is hardest.

Mask-first pipelines without healing

Even when graphs are built, gaps under canopy break connectivity. Most workflows do not try to reconnect likely-continuous road segments across small occlusions.

Static analysis tools

Some tools compute network metrics offline, but they do not tie gatekeeper ranking to real satellite tiles or let users simulate failures on the actual road layout.

Generic global models

Many models are trained for clear roads in open terrain. They are not designed for dense Indian urban grids with heavy tree cover, shadow, and irregular side streets.

Our difference

We train with occlusion-aware augmentation, using synthetic shadows and dropped patches so the model learns to segment roads under canopy, not just on open asphalt. We then repair broken topology at the graph stage, rank critical junctions on the healed network, and present everything in a single planner-facing workflow aligned with NNRMS urban road asset management.

USP of the proposed solution

End-to-end pipeline in one system

Satellite tile to road mask to healed graph to gatekeeper ranking to live dashboard. No handoff between disconnected tools.

Occlusion-aware by design

Training simulates canopy and shadow effects. Phase II explicitly repairs broken topology instead of discarding fragmented masks.

Graph-theoretic criticality, not just map coloring

Junctions are ranked using betweenness centrality on the actual road network, with stress tests for flood, accident, and construction scenarios.

Planner-ready output

Results are not buried in notebooks. The React dashboard lets users explore heatmaps, inspect gatekeepers, and simulate node failure with before-and-after network impact.

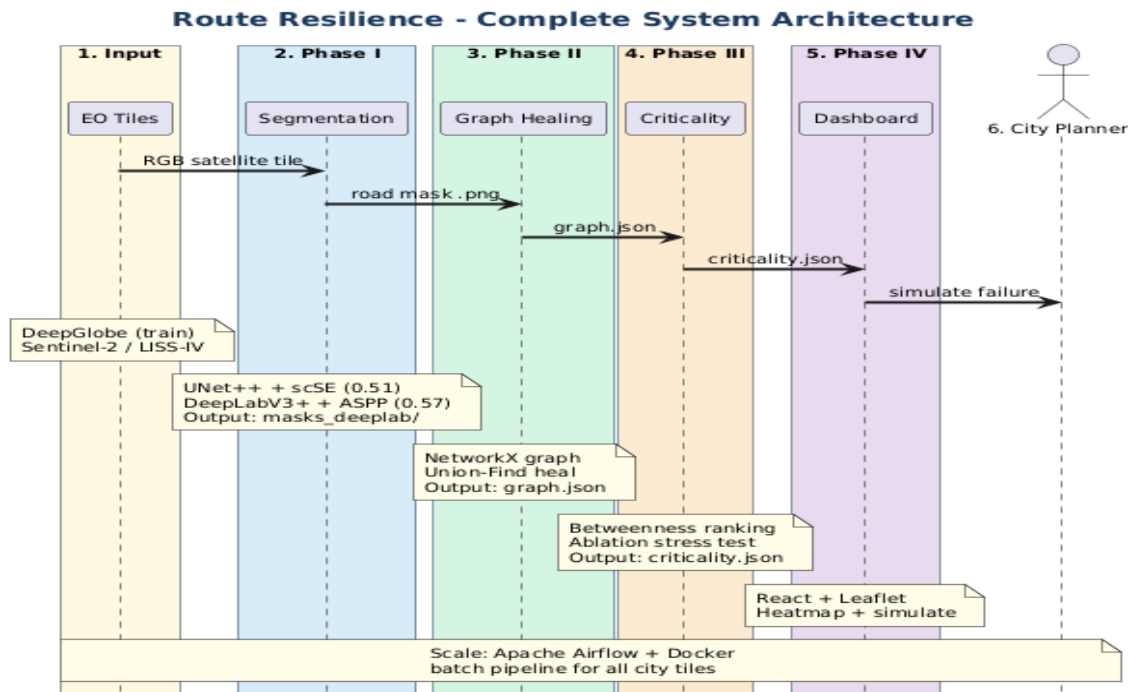
Built for Indian urban EO context

The approach targets tree-covered city grids and aligns with disaster planning, GIS asset mapping, and mobility resilience use cases under NNRMS.

Scalable execution path

The same pipeline can move from research notebooks to batch processing with Docker and Apache Airflow for city-wide refresh on ISRO-compatible EO tiles.

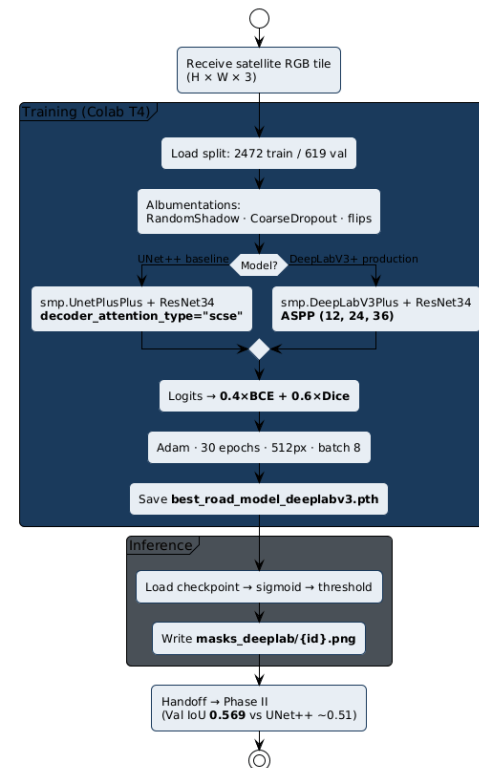
Architecture diagram for the proposed solution



Phase-1

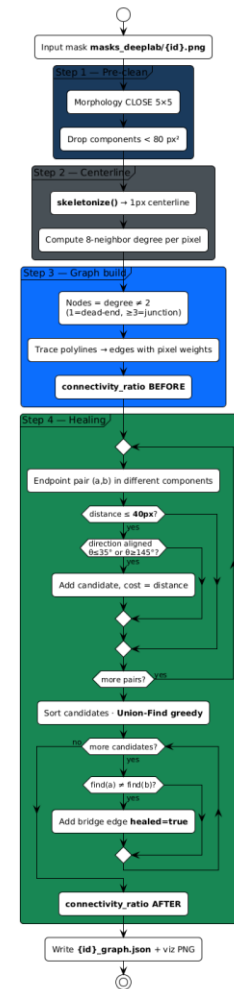
- Train occlusion-robust segmentation on satellite RGB tiles
- Baseline UNet++ with scSE attention; production DeepLabV3+ with ASPP
- Augmentation simulates shadow and missing patches under canopy
- Outputs a binary road mask for each tile
- Benchmark both architectures on the same data and loss before choosing the production model for batch inference

Phase I — Occlusion-Robust Road Segmentation



Phase-2

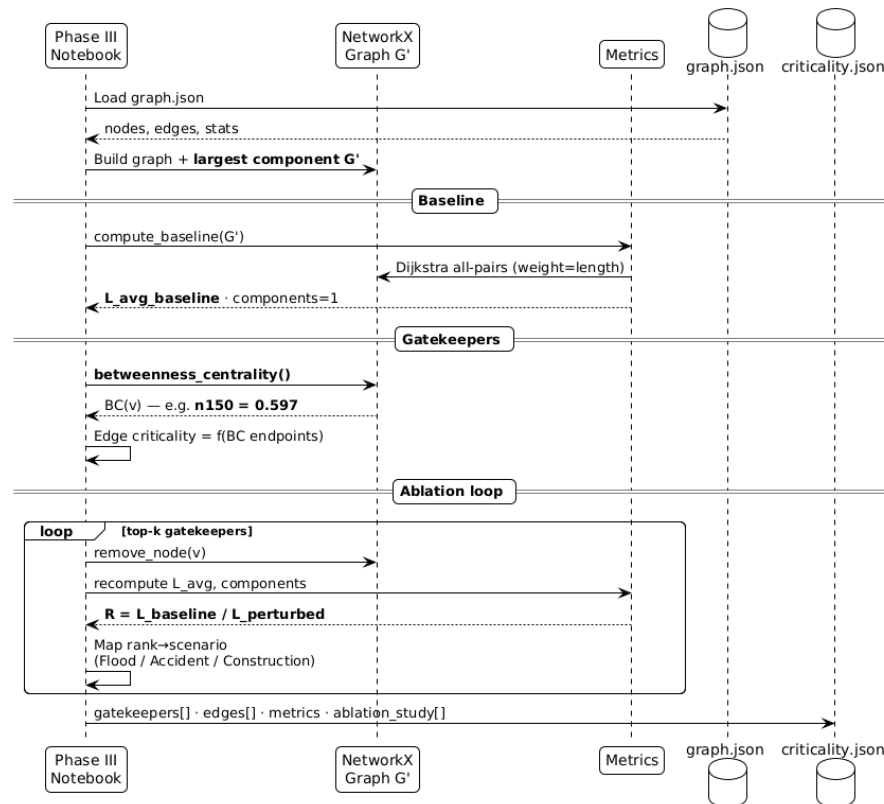
- Clean the road mask with morphological closing and small-component removal
- Skeletonize to a 1px centerline and compute per-pixel neighbor degree
- Build a graph with junctions and dead-ends as nodes and traced polylines as weighted edges
- Heal gaps by bridging nearby endpoints with Union-Find greedy matching
- Compare connectivity ratio before vs after and export the graph JSON plus a viz PNG



Phase-3

- Analyze the largest connected component of the road graph
- Rank junctions by betweenness centrality to find gatekeepers
- Score edges to build the criticality heatmap
- Stress-test by removing top nodes one at a time, mapped to flood, accident, and construction scenarios
- Compute a resilience score from the rise in average travel cost and save the criticality analysis file

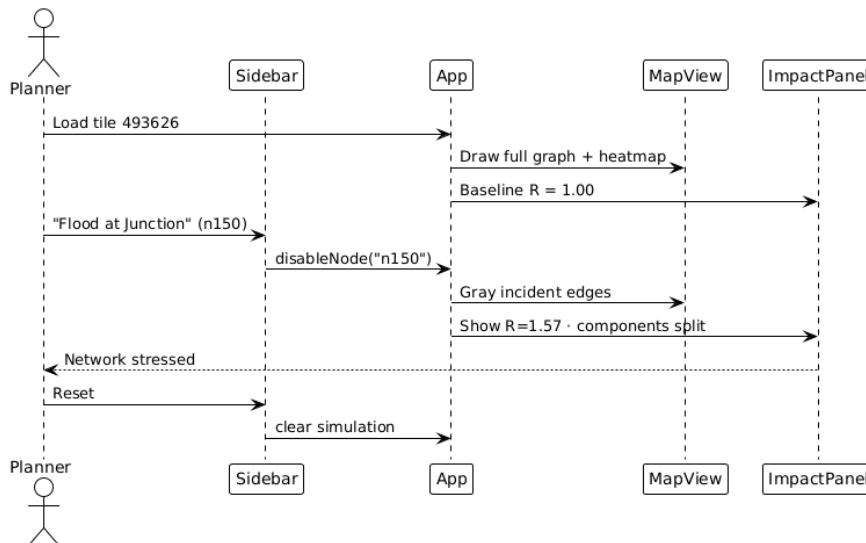
Phase III — Network Criticality & Stress Testing



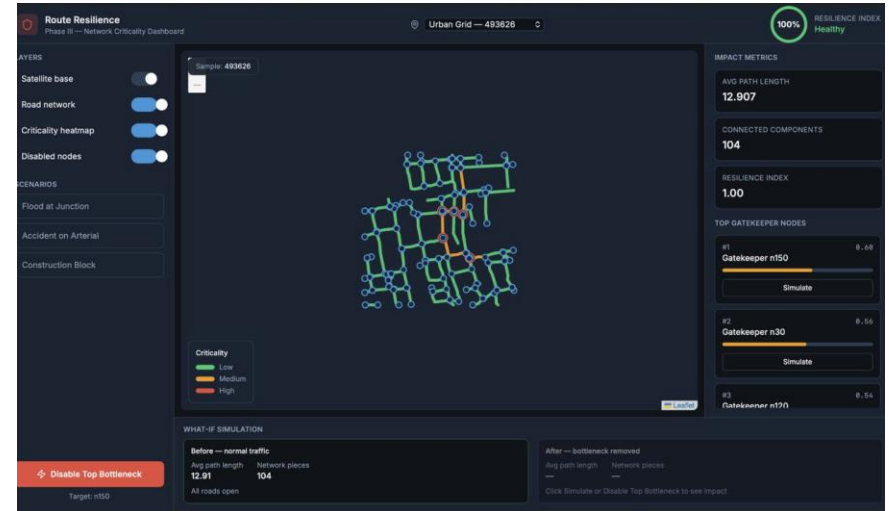
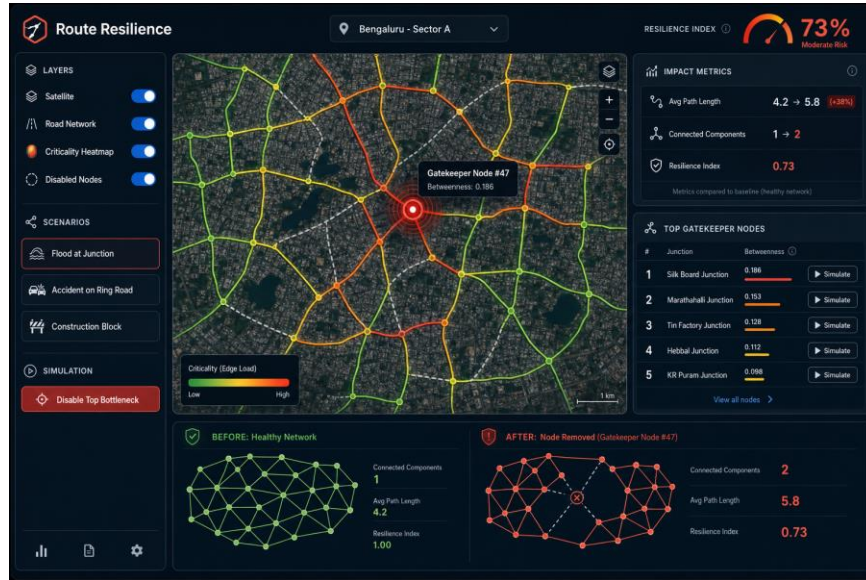
Phase-4

- Load the criticality data into a React dashboard with a Leaflet map
- Show the road network, heatmap, and gatekeeper markers as toggleable layers
- Select a scenario or click a node to simulate failure
- View before-and-after network metrics on the same screen
- Compare the healthy network versus the stressed network in a bottom panel

Phase IV — Simulate Failure



Wireframe/mockup frontend

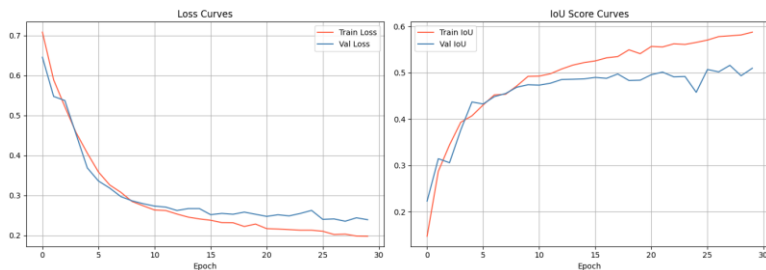


List of features offered by the solution

- Occlusion-robust road extraction from high-resolution satellite imagery
- Training with shadow and canopy simulation for real urban conditions
- Model comparison between attention-based and multi-scale segmentation architectures
- Binary road mask generation for city-scale tile processing
- Mask cleaning and centerline extraction for accurate network representation
- Automatic conversion of road pixels into a routable graph
- Topological repair of broken links caused by tree cover and shadow gaps
- Weighted road segments with junction and endpoint detection
- Identification of gatekeeper junctions using network centrality analysis
- Edge-level criticality scoring for heatmap visualization
- Stress testing through simulated junction or road failure
- Scenario mapping for flood, accident, and construction disruption
- Resilience assessment showing network degradation under failure
- Interactive web dashboard for exploration and presentation
- Map overlay of road network with criticality heatmap
- Layer controls for satellite view, roads, heatmap, and healed links
- Click-to-simulate failure on critical junctions
- Before-and-after comparison of network health metrics
- Ranked gatekeeper list with quick simulation actions
- End-to-end workflow from satellite input to planner decision support
- Designed for Indian urban grids with heavy canopy and irregular street layouts
- Scalable batch execution path for city-wide analysis on ISRO-compatible EO data

Phase I: Road extraction

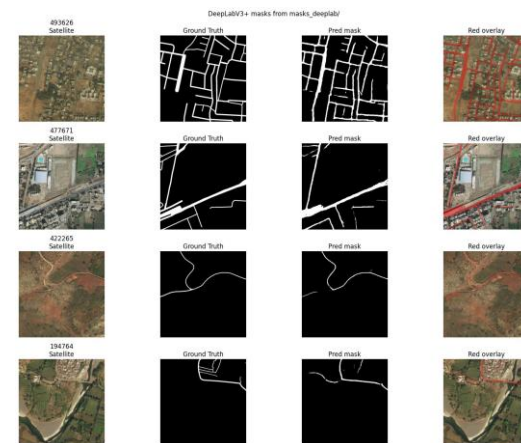
- Trained on DeepGlobe with occlusion augmentation (shadow + dropout)
- Baseline: **UNet++ with scSE** — Val IoU 0.51
- Production model: **DeepLabV3+ with ASPP** — Val IoU 0.569
- Batch inference on 619 validation tiles
- Best visual result: tile 493626 (urban grid)
- Weakest tile: 194764 (village/river — poor visibility)



UNet++ (scSE)
Baseline · Val IoU 0.51



DeepLabV3+ (ASPP)
Production · Val IoU 0.569

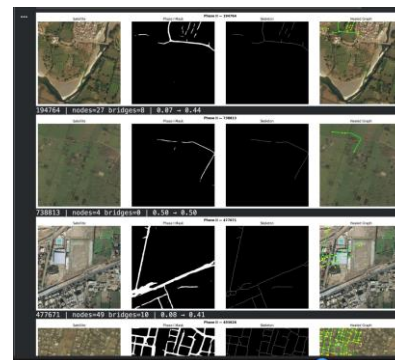


Phase II & III: Graph healing & criticality

Phase II — Graph + healing

Satellite, DeepLab mask, skeleton, and healed graph (green roads, red bridges, yellow nodes).

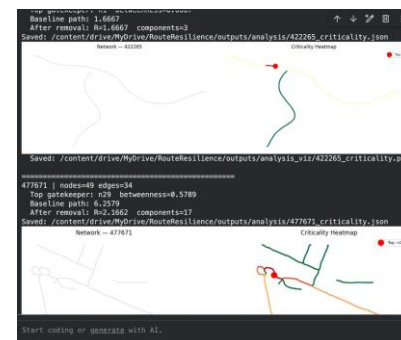
- **493626 (urban demo):** 645 nodes, 106 bridges, connectivity 0.01 $\rightarrow$ 0.06.
- **477671 (highway):** 121 nodes, 22 bridges, connectivity 0.03 $\rightarrow$ 0.26 — largest gain.
- **422265 (rural):** 13 nodes, 4 edges, no bridges, connectivity stays 0.15.
- **194764 (river/village):** 27 nodes, 8 bridges, connectivity 0.07 $\rightarrow$ 0.44.



Phase III — Criticality analysis

Road skeleton + criticality heatmap (green low $\rightarrow$ red high); top gatekeeper marked red.

- **493626 (demo):** 281 nodes, 177 edges; gatekeeper n150 (0.597); avg path 12.91; remove n150 $\rightarrow$ R 1.57, 107 components.
- **477671 (highway):** 49 nodes, 34 edges; n29 (0.579); avg path 6.26; R 2.17, 17 components.
- **422265 (rural):** 6 nodes, 4 edges; n1 (0.667); avg path 1.67; R 1.67, 3 components.
- **194764 (river/village):** 27 nodes, 21 edges; n23 (0.655); avg path 3.55; R 1.18, 8 components.



Implementation Cost

Phase I — GPU

30-epoch training plus batch inference on an NVIDIA T4 GPU (Google Colab).

Phases II–IV — CPU

Graph healing, criticality analysis, and dashboard build run on CPU with no extra training.

Total Cost — ₹0

Free-tier Colab GPU plus open-source tools only — zero direct cost for the prototype.

BHARATIYA
ANTARIKSH
HACKATHON

2026

THANK YOU